

Programação assíncrona com Python

Desde sua versão 3.4 Python possui integrada ao seu core a biblioteca `asyncio`, que permite a construção de código assíncrono não bloqueante. Nesse video vamos entender um pouco como essa biblioteca funciona com um breve exemplo.

Nos últimos 10 anos a programação assíncrona não bloqueante tem ganhado destaque no mercado de desenvolvimento por conta da diferença de performance entre tecnologias que possuem esse recurso, como Node.JS, e as que não possuem. Dado esse cenário, a comunidade Python demonstrou interesse no assunto de programação assíncrona não bloqueante e no ano de 2012 a biblioteca **`asyncio`** foi introduzida ao core da linguagem com a PEP 3156.

A biblioteca `asyncio` fornece à linguagem recursos para a criação de código concorrente sem precisar recorrer ao uso de múltiplas threadings, utilizando event loops em seu lugar, utilizando uma sintaxe fácil de entender a primeira vista:

```
1 import asyncio
2
3 async def ola_mundo():
4     print('Olá ...')
5     await asyncio.sleep(1)
6     print('... Mundo!')
7
8 asyncio.run(ola_mundo())
```

No código acima vemos a declaração de uma co-rotina, função assíncrona que utiliza do par declarativo `async/await` para determinar seu comportamento, onde `async` indica que se trata de uma função não bloqueante e `await` indica que a função deverá esperar o a execução da linha em segundo plano sem bloquear o resto da aplicação. A partir da versão 3.7 do Python é possível executar uma co-rotina através do método `asyncio.run()`.

A biblioteca `asyncio` fornece um conjunto de APIs de alto e baixo nível para a execução de co-rotinas paralelamente tendo total controle sobre sua execução, realizar operações de entrada e saída de rede e IPC, ter controle sobre subprocessos, utilizar filas para distribuir tarefas, sincronizar código que está executando de forma assíncrona.